abstracts the remote execution activity in higher level code chunks to control the remote machines intelligently, at scale, using an easy-to-learn modelling language.

The examples provided in this article are based on Ubuntu 14.04 LTS and MacOS X.

The traditional *NIX way through SSH

The basic structure of the *bash* command to remotely execute commands is shown below. The option *-tt* is required in case of the *sudo* command.

```
ssh -tt -oStrictHostKeyChecking=no user@<remote hostname that
is resolvable or IPv4 address> 'command(s) to run, separated
by ; or bunched together by && or || bash operators>'
```

This command presents a prompt to enter a password in case the remote login authentication requires it. If you use key-pair authentication, as is prevalent in the world of cloud based virtual machines, then you need to append *-i <private keypair file>* to the command as well. Typing the password for the remote authentication becomes cumbersome if you need to remote execute on a number of machines. You can use a utility known as *sshpass* to handle the password entry part through the command line, a file having that password or an environment variable. You can install *sshpass* through *sudo apt-get install -y sshpass* or compile that by downloading the tarball, extracting it and executing the command *sudo apt-get install -y build-essential && ./configure && sudo make install* in the source directory. The usage of *sshpass* with *ssh* is shown below, and you can see its *help* menu through *sshpass -h* to know more about the other options it provides:

```
sshpass -p '<password>' ssh <all the remaining options and
arguments>
```

The SSH commands need to be further wrapped in bash scripts to create home grown remote execution utilities that parse users, hostnames, IPs, commands, etc, from some configuration files and to run the remote execution in parallel. The advantage of this approach is that you don't need any extra dependency to start with, as bash is the de-facto option on almost every GNU/Linux distribution. So the development of the bash wrappers around the SSH command is a very quick way to start with. But the disadvantage of the bash approach is that the bash wrapper becomes complicated and hard to manage when you need to have a more intelligent tool. Features like retrying multiple times with some delay in between, skipping unresponsive machines, discovering machines at runtime, etc, complicate the tool development efforts a lot when using the raw SSH and bash approach.

The 'Python way' through Fabric

Job orchestration is the term very commonly used for so many functions in cloud environments. But I am using the term here to denote activities that are performed remotely in any cloud set-up. I'm denoting the activities to be performed on the remote VMs as jobs, and a job could consist of a very simple command or a complicated workflow of a number of simple/complicated commands. So when using the term 'job orchestration' in this article, I mean 'trigger and manage various related or unrelated remote and automated activities' to achieve some end result in a cloud set-up. For example, running configuration management tools jobs to provision VMs out-of-the-box, patching a group of VMs through remote jobs, monitoring/verifying VMs in an agentless manner through remote dump jobs, etc.

As mentioned in the earlier section, there is a need to use more general and user-friendly tools and frameworks rather than a raw bash based approach if we need to automate remote job execution in all kinds of cloud set-ups. Fabric is that kind of programmable framework that encapsulates all complications around using SSH to create jobs for automatic remote execution in a serial or parallel manner. Fabric is Python based and that means it can do everything possible through the data structures, constructs, standard libraries and external modules available in the language. Also, Python is one of the core components available, by default, in almost every GNU/Linux distribution. This means you only need to set up the Fabric components, and everything is set.

Fabric is a non-centralised framework. So you need to install it on one or more virtual/physical machines from where you need to trigger the jobs for your remote machines. The recommended way to install Fabric is to use the *pip* command on GNU/Linux and MacOS X. There is a version of Pip for Windows as well. You also need to download and install Python and Pycrypto (a version that matches the Python version) on Windows, from the links provided in the Reference section at the end of the article. The command to install Fabric using Pip is (remove the *sudo* part of the command on Windows):

```
sudo pip install -U fabric
```

Note: Based on my experience on Ubuntu 14.04 LTS, I would recommend installing Pip through *easy_install* using the command *sudo easy_install pip* and not through the Python-Pip package (run the command *sudo apt-get install -y python-setuptools* to get *easy_install*). Then run the command *sudo apt-get install -y python-dev build-essential && sudo pip install -U fabric* to build and install Fabric. Pip wasn't installed on my MacOS X laptop so I had to install it using *sudo easy_install pip* (*easy_install* was already available there). Also, installing Fabric through Pip failed on MacOS X initially due to the non-matching version of the Paramiko library. It required me to install the right version of Paramiko using the following command: